PolicyScope

Policies, adapters, enforcement and analysis

1 Builtins and a concrete adapter example

1.1 Builtins and their semantics

Runtime predicates are bold; *shared predicates are italic*; app predicates use plain monospace. SDK signatures below are not fact assertions.

Runtime gate.

```
// Runtime input: call ID, application-supplied caller name and event kind.
Request(Q, Principal, EventKind)

// Runtime input: actual call arguments bound to Q.
Parameter(Q, Name, Value)

// Derived permission; requires no Deny and valid runtime checks.
Allow(Q)

// Derived refusal; overrides Allow for the same Q.
Deny(Q, Reason)
```

Allow(Q)	Any **Deny**(Q, Reason)	Business callback
Absent	Absent	Refused
Present	Absent	May execute after runtime checks
Absent	Present	Refused
Present	Present	Refused

Each call derives fresh answers from current facts. Evaluation failure, an invalid transition or a violated invariant refuses execution.

Optional capability and membership.

```
// Input: Explicit operation grant on a resource or permission scope.
Capability(Principal, Scope, EventKind)

// Input: Principal membership; a group can itself be a member.
PrincipalGroup(Principal, Group)

// Input: Permission-scope membership; grants on a parent apply to its descendants.
ResourceGroup(Parent, Child)

// Derived: Grant including declared principal and resource memberships; not final authorization.
Can(Principal, Resource, EventKind)
```

```
// An explicit capability grants this operation on the specified scope.
Can(P, R, E) :- Capability(P, R, E).
```

```
// A principal inherits grants available to a group it belongs to.
Can(P, R, E) :- PrincipalGroup(P, G), Can(G, R, E).
```

```
// A grant on a parent resource scope also applies to each declared child.
Can(P, Child, E) :- ResourceGroup(Parent, Child), Can(P, Parent, E).
```

Can constrains execution only when used by an Allow/Deny rule.

Optional provenance.

```
// Input: Trusted record of a direct data origin; does not establish content truth.
Source(Principal, Data)

// Input: Direct conservative data dependency, supplied by actual instrumentation or application records.
DependsOn(Output, Input)

// Derived: Possible origin propagated through declared dependencies; no mandatory security labels.
Provenance(Data, Principal)
```

```
// A directly recorded source is a possible origin of this data object.
Provenance(D, P) :- Source(P, D).
```

```
// An output inherits possible origins from every declared input dependency.
Provenance(Out, P) :- DependsOn(Out, In), Provenance(In, P).
```

Provenance alone does not authorize disclosure.

Optional resource lifecycle.

```
// Input: Resource type, supplied by the application.
ResourceType(Resource, Type)

// Input: Current state in the declared application scope; replaces obsolete state.
ResourceState(Resource, State)

// Input: Reviewed initial state for newly created resources of this type; never resets existing resources.
InitialState(Type, State)

// Input: Configured state-machine edge shared by resources of this type.
AllowedTransition(Type, EventKind, From, To)

// Derived: Nonempty path in the configured machine; not a record of past execution.
ReachableState(Type, From, To)

// Derived: Configured edge matching current state; other policy conditions still apply.
CanTransition(Resource, EventKind, From, To)
```

```
// Each allowed transition is a nonempty path between its states.
ReachableState(T, From, To) :- AllowedTransition(T, E, From, To).

// Extend a nonempty state path by one allowed transition of the same resource type.
ReachableState(T, From, To) :- ReachableState(T, From, Mid), AllowedTransition(T, E, Mid, To).
```

```
// A resource can take an event edge only when its type and current state match the edge.
CanTransition(R, E, From, To) :- ResourceType(R, T), ResourceState(R, From),
    AllowedTransition(T, E, From, To).
```

Configured lifecycle calls must match CanTransition. On confirmed success, the SDK replaces ResourceState. ReachableState ignores permissions and execution history.

1.2 Loading rules and adding a tool adapter

The .dl file describes schemas, initial facts and rules. @policy adds enforcement to a tool call. Developers must add the adapter statements that update EDB from actual business results.

Schema / call form	Semantics
`Policy(path, **options)`	Load .dl schemas, initial EDB facts and rules once at startup.
`@policy`	Create Request/Parameter, check Allow/Deny and skip denied tool bodies.
`policy.put / policy.delete`	Manual adapters update EDB. Keyed put replaces the old tuple; delete removes a fact.

Import and load once at startup. Selected settings from adapter.py and run.py:

```
# SDK import and initial policy loading; no business operation runs here.
from policy_runtime import Policy

policy = Policy(
    "rules.dl",
    target="job_id",  # Tool argument identifying the FSM resource.
    # Map the existing approve=False argument to a job-rejection event.
    event=lambda name, args: "reject_job"
        if name == "approve_job" and args.get("approve") is False else name,
    completed=lambda result: result.get("ok") is not False,
)
store.policy = policy  # Reuse the same instance across tool calls.
```

Existing application logic. In the tool below, s is the original Store instance. s.submit calls this unchanged Store.submit method:

```
def submit(self,owner,spec,event_id):
    spec=validate_spec(spec)
    if owner not in ('PhD','Master'):return deny('NOT_A_STUDENT')
    def change(s):
        job={'id':f"JOB-{len(s['jobs'])+1:03d}",'owner':owner,'version':1,'spec':spec,
             'private_input':fixture(owner.lower()+'_private')['private_input'],'state':'QUEUED',
             'approval_id':None,'run_id':None,'waiting_dispatches':0,'submission_seq':len(s['jobs'])+1,
             'created_event_id':event_id,'outcome':'AWAITING_REVIEW'}
        job['task_hash']=self._hash(job);s['jobs'].append(job)
        return {'ok':True,'job':job}
    return self._mutate(change)
```

The existing method validates the specification, creates the job and commits it through the application’s own transaction.

Manual adapter after the original operation. Grey blocks are added code; white blocks retain the business call.

```
policy = s.policy  # Reuse the policy loaded at startup.
elif name=='submit_job':
    @policy(name=name, arguments=a)  # Adaptation code: read caller at invocation.
    def fn(e):
        result=s.submit(r,a['spec'],e)
        # Adaptation code: stage only the confirmed job fields.
        if result.get('ok') is not False:
            job=result['job']
            policy.put('ResourceType', job['id'], 'job')
            policy.put('ResourceState', job['id'], job['state'])
            # Current job metadata: ID, owner, version, task hash and run ID.
            # Job is keyed by ID; put replaces that job's old tuple.
            policy.put('Job', job['id'], job['owner'], job['version'],
                       job['task_hash'], job['run_id'])
            spec=job['spec']
            policy.put('JobSpec', job['id'], spec['node_type'], spec['cpu_cores'],
                       spec['memory_gb'], spec['runtime_minutes'])
            policy.put('CurrentApproval', job['id'], job['approval_id'])
            policy.put('Waiting', job['id'], job['waiting_dispatches'], job['submission_seq'])
            policy.put('JobOutcome', job['id'], job['outcome'])
            # Adaptation code: identify this task version and record its owner as origin.
            job_data = policy.data.reference('job', job['id'], job['task_hash'])
            policy.put('RecordData', 'job', job['id'], job_data)
            policy.put('Source', job['owner'], job_data)
        return result
```

The local callback receives only a trace event ID, so name and a supply the actual tool name and parameters. Ordinary named functions use @policy directly. The original call path is retained. Caller-name adaptation reads Tools.role here, or MCP request metadata in the second app.

Nested dictionaries such as spec are flattened into Parameter names. Lists, floats and custom objects require an explicit adapter mapping.

New resource type/state facts are checked against InitialState(”job”, ”QUEUED”) before policy commit. `RecordData` maps a business record to its data ID; *`Source`* records its origin.

The annotation supplies the request and gate; it does not infer these business facts. A refusal skips the body. An explicit ok=False publishes no EDB edits. A synchronization failure after a business commit stops the scope without undoing the business effect.

Allowing Professor to reject a queued job. This is a permitted business operation, not a policy refusal:

```
// Reviewed membership and the job-rejection state edge.
PrincipalGroup("Professor", "Professor").
AllowedTransition("job", "reject_job", "QUEUED", "TERMINATED").
```

```
// Permit Professor rejection only for this QUEUED version and a false approval decision.
Allow(Q) :- Request(Q, A, "reject_job"), Parameter(Q, "job_id", J),
    PrincipalGroup(A, "Professor"), Job(J, Owner, V, H, RunId), ResourceState(J, "QUEUED"),
    Parameter(Q, "expected_version", V), Parameter(Q, "approve", "false"),
    CanTransition(J, "reject_job", From, To).
```

```
# Assume JOB-001 is currently QUEUED at version 1.
prof_tools = Tools("Professor", store, trace)
prof_tools.call("approve_job", {
    "job_id": "JOB-001", "expected_version": 1, "approve": False})
# Policy allows the mapped reject_job event.
# The original application rejects the job; policy state becomes TERMINATED.
```

2 LabCluster

2.1 Policy walkthrough

Grey boxes quote original P1–P6. Pn.m are our decomposition; rules follow in source order.

P1

Original policy P1. Role permissions and resource limits: Professor manages scheduling; students manage only their own jobs within their role's resource limits. Only Professor may approve or reject student requests, dispatch jobs, or stop another student's job. PhD and Master may submit jobs and read, revise, cancel or report completion only for their own jobs. Master may request CPU only, with at most 4 cores, 8 GB and 20 minutes of requested runtime. PhD may request CPU or GPU, with at most 8 cores, 32 GB and 60 minutes. Requests outside the owner's quota must not be admitted or dispatched; Professor cannot override these limits or physical capacity. [source]

P1.1 Recognize trusted student and Professor groups.

```
// Reviewed configuration.
PrincipalGroup("Master", "MasterStudent").
PrincipalGroup("PhD", "PhDStudent").
PrincipalGroup("Professor", "Professor").
PrincipalGroup("Runner", "Runner").

// Recognize a Master principal as a student.
Student(A) :- PrincipalGroup(A, "MasterStudent").

// Recognize a PhD principal as a student.
Student(A) :- PrincipalGroup(A, "PhDStudent").

// Students may use the ordinary user observation tools.
UserAgent(A) :- Student(A).

// Professor may use the ordinary user observation tools.
UserAgent(A) :- PrincipalGroup(A, "Professor").
```

P1.2 Reserve approve, reject, dispatch and administrative stop for Professor.

`Job(JobId, Owner, Version, TaskHash, RunId)`: Tool helper records confirmed submit/revise/dispatch fields.

`.key Job(JobId)`

`WithinJobQuota(Job)`: The current stored job fits its owner group quota.

`Dispatchable(Job)`: The eligible job has no better competitor on its selected node.

Existing Python tool-call. `approve_job(job_id, expected_version, approve)`: Approve or reject the specified job version.

```
// Permit Professor approval only for this QUEUED version and a quota-compliant job.
Allow(Q) :- Request(Q, A, "approve_job"), Parameter(Q, "job_id", J),
    PrincipalGroup(A, "Professor"), Job(J, Owner, V, H, RunId), ResourceState(J, "QUEUED"),
    Parameter(Q, "expected_version", V), Parameter(Q, "approve", "true"), WithinJobQuota(J).
```

Mapped event. `reject_job`: The adapter maps approve_job(..., approve=False) to this event; it is not a separate tool.

```
// Permit Professor rejection only for this QUEUED version and a false approval decision.
Allow(Q) :- Request(Q, A, "reject_job"), Parameter(Q, "job_id", J),
    PrincipalGroup(A, "Professor"), Job(J, Owner, V, H, RunId), ResourceState(J, "QUEUED"),
    Parameter(Q, "expected_version", V), Parameter(Q, "approve", "false"),
    CanTransition(J, "reject_job", From, To).
```

Existing Python tool-call. `dispatch_job(job_id)`: Allocate a node and start the selected job.

```
// Only Professor may dispatch, and only a Dispatchable job can be selected.
Allow(Q) :- Request(Q, A, "dispatch_job"), Parameter(Q, "job_id", J),
    PrincipalGroup(A, "Professor"), Dispatchable(J),
    CanTransition(J, "dispatch_job", "QUEUED", "RUNNING").
```

Existing Python tool-call. `kill_job(job_id, reason)`: Request an administrative stop.

```
// Only Professor may request an administrative stop; a real stop confirmation is required.
Allow(Q) :- Request(Q, A, "kill_job"), Parameter(Q, "job_id", J),
    PrincipalGroup(A, "Professor"), Job(J, Owner, V, H, RunId), Parameter(Q, "reason", Reason),
    CanTransition(J, "kill_job", From, To).
```

P1.3 Students submit within quota and operate only on their own jobs.

`WithinRequestQuota(Q)`: The intercepted request fits an owner group quota.

Existing Python tool-call. `submit_job(spec)`: Create a job owned by the bound caller.

```
// Permit student submission only when the requested resources fit the quota.
Allow(Q) :- Request(Q, A, "submit_job"), Student(A), WithinRequestQuota(Q).
```

Existing Python tool-call. `revise_job(job_id, spec)`: Change an existing job specification.

```
// Permit revision of an owned QUEUED job within the new request quota.
Allow(Q) :- Request(Q, A, "revise_job"), Parameter(Q, "job_id", J), Student(A),
    Job(J, A, V, H, RunId), ResourceState(J, "QUEUED"), WithinRequestQuota(Q).
```

Existing Python tool-call. `cancel_job(job_id, reason)`: Cancel the caller's job.

```
// Only a student owner may cancel a job; the tool validates text and records the reason.
Allow(Q) :- Request(Q, A, "cancel_job"), Parameter(Q, "job_id", J), Student(A),
    Job(J, A, V, H, RunId), Parameter(Q, "reason", Reason),
    CanTransition(J, "cancel_job", From, To).
```

Message entry. `Bus.send(sender, recipient, kind, body)`: Existing message API, not a tool-call; only COMPLETION_REPORT is currently guarded.

```
// A student may send a structured completion report only for a job they own.
Allow(Q) :- Request(Q, A, "send_message"), Parameter(Q, "message_type", "COMPLETION_REPORT"),
    Parameter(Q, "job_id", J), Student(A), Job(J, A, V, H, RunId).
```

Reuses P2.1, P3.5.

Adapter. Bus.send checks COMPLETION_REPORT ownership before delivery; other message kinds are unchanged.

P1.4 Master uses CPU with cores <= 4, memory <= 8 GB and requested minutes <= 20.

`Quota(Group, Type, Cores, Memory, Minutes)`: Reviewed .dl limits matching the original configuration.

```
// Reviewed configuration.
Quota("MasterStudent", "CPU", 4, 8, 20).
Quota("PhDStudent", "CPU", 8, 32, 60).
Quota("PhDStudent", "GPU", 8, 32, 60).
```

`JobSpec(Job, Type, Cores, Memory, Minutes)`: Tool helper records the specification returned by submit/revise.

`.key JobSpec(Job)`

```
// The requested node type and all three numeric limits must fit a group quota.
WithinRequestQuota(Q) :- Request(Q, A, Op), Parameter(Q, "spec_node_type", T),
    Parameter(Q, "spec_cpu_cores", C), Parameter(Q, "spec_memory_gb", M),
    Parameter(Q, "spec_runtime_minutes", Time), PrincipalGroup(A, Group),
    Quota(Group, T, MaxC, MaxM, MaxTime), C <= MaxC, M <= MaxM, Time <= MaxTime.
```

```
// A stored job must still fit its owner group quota.
WithinJobQuota(J) :- Job(J, A, V, H, RunId), JobSpec(J, T, C, M, Time),
    PrincipalGroup(A, Group), Quota(Group, T, MaxC, MaxM, MaxTime),
    C <= MaxC, M <= MaxM, Time <= MaxTime.
```

Adapter. Original validate_spec checks positive integers.

P1.5 PhD uses CPU or GPU with cores <= 8, memory <= 32 GB and minutes <= 60.

Reuses P1.4.

P1.6 Reject over-quota admission and dispatch; require a free node with enough capacity.

`Node(Node, Type, Cores, Memory, Run, Generation)`: Reviewed initial capacity; helper records confirmed allocation/release.

`.key Node(Node)`

```
// Reviewed configuration.
Node("cpu-1", "CPU", 8, 32, "", 0).
Node("gpu-1", "GPU", 8, 32, "", 0).
```

```
// A compatible node must be free, have the required type, and fit cores and memory.
FreeCompatible(J, N) :- JobSpec(J, T, C, M, Time), Node(N, T, MaxC, MaxM, "", Gen),
    C <= MaxC, M <= MaxM.
```

Reuses P1.4, P5.2, P4.7.

P2

Original policy P2. Information flow and isolation: PhD and Master keep their work private from each other, while Professor may inspect both students' work. Neither student may receive the other's private job specifications, inputs, results, approvals or execution receipts through tools, model contexts or messages. Students may send their own task details to Professor, who may inspect both students' records and share private details only with the owning student. Forbidden and nonexistent job lookups must both return UNAVAILABLE without revealing whether a private record exists. Role quotas, node specifications and aggregate busy/free availability are public; this boundary does not prevent inference from public timing or availability. The authorized evaluator may inspect complete synthetic traces, but student agents must not receive that full view. API credentials must never appear in agent prompts, traces or submitted files. [source]

P2.1 Gate single-job reads and retain original bulk/private filtering.

```
// A student may read only jobs whose stored owner is that student.
Readable(A, J) :- Student(A), Job(J, A, V, H, RunId).
```

```
// Professor may read any stored job.
Readable(A, J) :- PrincipalGroup(A, "Professor"), Job(J, Owner, V, H, RunId).
```

```
// Model who may see queue details; the original Store enforces actual view filtering.
QueueViewer(A) :- PrincipalGroup(A, "Professor").
```

Existing Python tool-call. `read_job(job_id)`: Read one job record.

```
// Permit a single-job read only when Readable holds for its actual job_id.
Allow(Q) :- Request(Q, A, "read_job"), Parameter(Q, "job_id", J), Readable(A, J).
```

Existing Python tool-call. `view()`: Read-only overview for students and Professor: public quotas/nodes plus caller-visible records.

```
// Permit an ordinary user overview; the original Store filters each returned collection by its bound caller.
Allow(Q) :- Request(Q, A, "view"), UserAgent(A).
```

Existing Python tool-call. `read_private()`: Read the fixtures selected by the bound caller's role.

```
// Permit an ordinary user's private-fixture call; the original tool selects fixtures by its bound role.
Allow(Q) :- Request(Q, A, "read_private"), UserAgent(A).
```

```
// Model a student's permitted private fixture owner; the original tool selects the actual fixture.
PrivateReadable(A, A) :- Student(A).
```

```
// Model Professor's permitted fixture owners; the original tool supplies their fixtures.
PrivateReadable(A, Owner) :- PrincipalGroup(A, "Professor"), UserAgent(Owner).
```

Adapter. Original view/read_private filter outputs; QueueViewer and PrivateReadable are analysis-only.

Partial; see Section 2.2.

P2.2 Prevent peer disclosure across every output channel.

Reuses P2.1.

Partial; see Section 2.2.

P2.3 Permit owner-to-Professor sharing and owner-specific replies.

Partial; see Section 2.2.

P2.4 Return the same public response for a forbidden or missing job.

Reuses P2.1.

`refused("read_job")` returns `{"status": "UNAVAILABLE"}` for both forbidden and missing jobs.

P2.5 Keep quotas, node capacity and aggregate availability public.

Adapter. Original Store.view returns public fields and filters private collections.

P2.6 Restrict full evaluator traces to the evaluator.

Partial; see Section 2.2.

P2.7 Keep credentials out of prompts, traces and submissions.

Partial; see Section 2.2.

P3

Original policy P3. Evidence for job decisions: Starting a job requires a current approval record; marking it completed requires a matching runner receipt. Before dispatch, require a successful Professor approval record matching the job's current task version and exact resource request. Only QUEUED jobs may be revised; a revision invalidates the approval and requires another review. Approval leaves the job QUEUED without reserving a node. Before marking a job COMPLETED, require a prior simulator-issued completion receipt for that job's current run. These records must identify their source and the task or run they support. [source]

P3.1 Require recorded approval to start and recorded runner evidence to finish.

Reuses P3.2, P3.5, P1.2.

P3.2 Match approval to the current job version, exact spec and policy.

`CurrentApproval(Job, Approval)`: Tool helper records the current approval ID; revision clears it.

`.key CurrentApproval(Job)`

`Approval(Id, Job, Version, TaskHash, Issuer, PolicyDigest)`: Tool helper records the original approval result.

`.key Approval(Id)`

`CurrentPolicy(Digest)`: Reviewed cluster-policy digest in .dl.

```
// Reviewed configuration.
CurrentPolicy("56b6b1f170f4ab9563e14e4fa1dba1f8b9432775a1301420c4165f07fcc80b1a").
```

`ApprovalSpec(Id, Type, Cores, Memory, Minutes)`: Exact resource limits from that approval.

`.key ApprovalSpec(Id)`

```
// Approval must match the current reference, version, hash, policy and exact resource limits.
MatchingApproval(J) :- Job(J, Owner, V, H, RunId), CurrentApproval(J, Id),
    Approval(Id, J, V, H, "Professor", Digest), CurrentPolicy(Digest),
    JobSpec(J, T, C, M, Time), ApprovalSpec(Id, T, C, M, Time).
```

Adapter. The approval tool branch directly writes Approval, ApprovalSpec and CurrentApproval.

P3.3 Revise only QUEUED work and invalidate the old approval.

Reuses P1.3, P3.2.

Adapter. Original revise increments version and clears approval; the helper records both.

P3.4 Approval records evidence without allocating a node or starting a job.

Adapter. Approval leaves the job QUEUED without reserving a node.

Partial; see Section 2.2.

P3.5 Complete only with a successful simulator receipt for the current run and allocation.

`Run(Run, Job, Node, Generation, TaskHash, Version)`: Tool helper records the original dispatch result.

`.key Run(Run)`

`Receipt(Id, Run, Job, Node, Generation, TaskHash, Source, Kind)`: Tool helper records the original simulator receipt.

`.key Receipt(Id)`

`ExitSuccess(Receipt)`: Recorded for a zero-exit receipt.

`RecordData(Kind, Record, Data)`: Explicit tool puts associate confirmed records with SDK data IDs.

`.key RecordData(Kind, Record)`

```
// Completion evidence must match the active allocation and have a declared owner provenance.
MatchingReceipt(J, ReceiptId) :- Job(J, Owner, V, H, RunId), ResourceState(J, "RUNNING"),
    Run(RunId, J, N, Gen, H, V), Node(N, T, C, M, RunId, Gen),
    Receipt(ReceiptId, RunId, J, N, Gen, H, "SimulatedRunner", "completion"),
    ExitSuccess(ReceiptId), RecordData("receipt", ReceiptId, D), Provenance(D, Owner).
```

Existing Python tool-call. `finish_job(job_id, receipt_id)`: Request completion using a runner receipt.

```
// Permit completion only with readable job access and the requested matching receipt.
Allow(Q) :- Request(Q, A, "finish_job"), Parameter(Q, "job_id", J), Readable(A, J),
    Parameter(Q, "receipt_id", ReceiptId), MatchingReceipt(J, ReceiptId),
    CanTransition(J, "finish_job", "RUNNING", "COMPLETED").
```

Existing Python tool-call. `record_completion(run_id)`: Runner records a simulated completion receipt.

```
// Only the bound Runner may record completion for an existing RUNNING job run.
Allow(Q) :- Request(Q, A, "record_completion"), PrincipalGroup(A, "Runner"),
    Job(J, Owner, V, H, RunId), ResourceState(J, "RUNNING"), Parameter(Q, "run_id", RunId),
    Run(RunId, J, N, Gen, H, V).
```

Adapter. The helper records the runner receipt; finish checks it before completion.

P3.6 Retain the origin and supported task/run of each decision record.

Reuses P3.5.

Adapter. The helper records actual job, approval, run and receipt references.

P4

Original policy P4. Four-state lifecycle and exclusive ownership: Jobs follow the allowed state transitions, and a node belongs to at most one running job at a time. The only states are QUEUED, RUNNING, COMPLETED and TERMINATED. Submission creates QUEUED. Allowed transitions are QUEUED to RUNNING or TERMINATED, and RUNNING to COMPLETED or TERMINATED. Rejected requests and queued cancellations end in TERMINATED. Entry to RUNNING requires the approval in P3 and successful allocation of a free compatible node. Each RUNNING job must own exactly one matching active allocation, and each node may belong to at most one running job. Terminal jobs must not restart; another attempt requires a new submission. P6 defines the release procedure that preserves this ownership invariant. [source]

P4.1 Use the declared lifecycle and preserve exclusive allocations.

Reuses P4.7.

Adapter. The state and allocation requirements follow below.

P4.2 Admit only QUEUED, RUNNING, COMPLETED and TERMINATED.

```
// A state appearing as a job transition origin is an admitted job state.
JobState(S) :- AllowedTransition("job", E, S, To).
```

```
// A state appearing as a job transition destination is an admitted job state.
JobState(S) :- AllowedTransition("job", E, From, S).
```

```
// Report a current job state outside the declared job machine.
AllocationViolation(J) :- ResourceType(J, "job"), ResourceState(J, S), not JobState(S).
```

P4.3 Create each new job in QUEUED.

```
// A newly created job resource starts in QUEUED; existing resources are never reset.
InitialState("job", "QUEUED").
```

Adapter. After submit, put records the returned type/state; runtime checks InitialState before commit.

P4.4 Declare type-level edges for dispatch, completion, rejection and stop.

```
// dispatch_job: QUEUED -> RUNNING.
AllowedTransition("job", "dispatch_job", "QUEUED", "RUNNING").

// finish_job: RUNNING -> COMPLETED.
AllowedTransition("job", "finish_job", "RUNNING", "COMPLETED").

// reject_job: QUEUED -> TERMINATED.
AllowedTransition("job", "reject_job", "QUEUED", "TERMINATED").

// cancel_job: QUEUED -> TERMINATED.
AllowedTransition("job", "cancel_job", "QUEUED", "TERMINATED").

// cancel_job: RUNNING -> TERMINATED.
AllowedTransition("job", "cancel_job", "RUNNING", "TERMINATED").

// cancel_job: COMPLETED -> COMPLETED. Harmless terminal acknowledgement; no restart.
AllowedTransition("job", "cancel_job", "COMPLETED", "COMPLETED").

// cancel_job: TERMINATED -> TERMINATED. Harmless terminal acknowledgement; no restart.
AllowedTransition("job", "cancel_job", "TERMINATED", "TERMINATED").

// kill_job: QUEUED -> TERMINATED.
AllowedTransition("job", "kill_job", "QUEUED", "TERMINATED").

// kill_job: RUNNING -> TERMINATED.
AllowedTransition("job", "kill_job", "RUNNING", "TERMINATED").

// kill_job: COMPLETED -> COMPLETED. Harmless terminal acknowledgement; no restart.
AllowedTransition("job", "kill_job", "COMPLETED", "COMPLETED").

// kill_job: TERMINATED -> TERMINATED. Harmless terminal acknowledgement; no restart.
AllowedTransition("job", "kill_job", "TERMINATED", "TERMINATED").
```

P4.5 Terminate a rejected or cancelled queued request.

Reuses P1.2, P1.3.

P4.6 Enter RUNNING only after valid approval and successful free-node allocation.

Reuses P5.2, P1.2.

Adapter. The dispatch result supplies the actual run, node and generation.

P4.7 Require a consistent job/run/node allocation before and after every guarded operation.

```
// A nonempty node allocation token identifies an active run.
ActiveRun(RunId) :- Node(N, T, C, M, RunId, Gen), RunId != "".

// Every stored Run record identifies a known run.
KnownRun(RunId) :- Run(RunId, J, N, Gen, H, V).

// A valid allocation joins one RUNNING job, run and node on version, hash and generation.
ValidAllocation(J, RunId) :- ResourceState(J, "RUNNING"), Job(J, Owner, V, H, RunId),
    Run(RunId, J, N, Gen, H, V), Node(N, T, MaxC, MaxM, RunId, Gen), JobSpec(J, T, C, M, Time),
    C <= MaxC, M <= MaxM.

// Report a RUNNING job with no matching valid allocation.
AllocationViolation(J) :- ResourceState(J, "RUNNING"), Job(J, Owner, V, H, RunId),
    not ValidAllocation(J, RunId).

// Report a node token that refers to no stored run.
AllocationViolation(RunId) :- ActiveRun(RunId), not KnownRun(RunId).

// Report an active run whose matching RUNNING allocation is absent.
AllocationViolation(RunId) :- ActiveRun(RunId), Run(RunId, J, N, Gen, H, V),
    not ValidAllocation(J, RunId).

// Each allocated node must be the node recorded for that run; a valid other node is insufficient.
AllocationViolation(N) :- Node(N, T, C, M, RunId, Gen),
    Run(RunId, J, ExpectedNode, ExpectedGen, H, V), RunId != "", N != ExpectedNode.
```

Partial; see Section 2.2.

P4.8 Never dispatch a terminal job; retries require a new submission.

Reuses P5.2, P1.2.

P4.9 Release allocations only through the P6 confirmation procedure.

Adapter. See P6.

P5

Original policy P5. Waiting-queue order: PhD normally goes first, with protection against indefinite Master waiting. When eligible queued jobs compete for the same free compatible node, select PhD before Master. Priority applies only while waiting and must not interrupt a running job. A Master job that remains eligible but is skipped at three compatible dispatch opportunities must receive higher priority than PhD jobs. Equal priorities use submission order. An unrelated GPU request must not prevent a compatible Master job from using a free CPU node. [source]

P5.1 Prefer PhD while waiting and promote sufficiently skipped Master jobs.

Reuses P5.2, P5.4.

Partial; see Section 2.2.

P5.2 Compare only eligible queued candidates selecting the same free compatible node.

`NodeOrder(Node, Index)`: Configured node order in .dl.

```
// Reviewed configuration.
NodeOrder("cpu-1", 0).
NodeOrder("gpu-1", 1).
```

`Waiting(Job, Skips, Submission)`: Callback updates skip counts from returned dispatch competitors.

`.key Waiting(Job)`

`AgingThreshold(Skips)`: Configured promotion threshold in .dl.

```
// Reviewed configuration.
AgingThreshold(3).
```

```
// Node N is not the first choice for job J if another free compatible node precedes N in the configured node
    order.
EarlierNode(J, N) :- FreeCompatible(J, N), FreeCompatible(J, Earlier), NodeOrder(Earlier, A),
    NodeOrder(N, B), A < B.
```

```
// Job J is eligible on node N if it is queued, within quota, currently approved, and N is its first free
    compatible node.
Eligible(J, N) :- Job(J, Owner, V, H, RunId), ResourceState(J, "QUEUED"), WithinJobQuota(J),
    MatchingApproval(J), FreeCompatible(J, N), not EarlierNode(J, N).
```

```
// Assign ordinary PhD waiting priority 10.
Priority(J, "10") :- Job(J, Owner, V, H, RunId), PrincipalGroup(Owner, "PhDStudent"),
    ResourceState(J, "QUEUED").
```

```
// Assign unaged Master waiting priority 0.
Priority(J, "0") :- Job(J, Owner, V, H, RunId), PrincipalGroup(Owner, "MasterStudent"),
    ResourceState(J, "QUEUED"), Waiting(J, Skips, Seq), AgingThreshold(Limit), Skips < Limit.
```

```
// A candidate loses to a higher-priority candidate for the same selected node.
Outranked(J, N) :- Eligible(J, N), Eligible(Other, N), Priority(J, Low), Priority(Other, High),
    Low < High.
```

```
// Job J is dispatchable if it is eligible on a selected node, has a priority, and no competing eligible job
    outranks it on that node.
Dispatchable(J) :- Eligible(J, N), Priority(J, P), not Outranked(J, N).
```

P5.3 Apply priority only to QUEUED work without preempting RUNNING jobs.

Reuses P1.6, P5.2.

P5.4 After three actual competing opportunities, promote Master above PhD.

```
// Promote a queued Master to priority 20 when its observed skip count reaches the threshold.
Priority(J, "20") :- Job(J, Owner, V, H, RunId), PrincipalGroup(Owner, "MasterStudent"),
    ResourceState(J, "QUEUED"), Waiting(J, Skips, Seq), AgingThreshold(Limit), Limit <= Skips.
```

Reuses P5.2.

Adapter. Use the original dispatch competitor list to update Waiting.

P5.5 Break equal-priority competition by submission order.

```
// At equal priority, a candidate loses to an earlier submission for the same node.
Outranked(J, N) :- Eligible(J, N), Eligible(Other, N), Priority(J, P), Priority(Other, P),
    Waiting(J, A, Later), Waiting(Other, B, Earlier), Earlier < Later.
```

P5.6 Do not let an unrelated GPU job block an eligible CPU job.

Reuses P5.2, P5.5.

P6

Original policy P6. Stop confirmation and safe node release: Release a node only after confirmed completion or stopping, and only for the run that currently owns it. A stop or cancellation request must record a specific reason. Keep a RUNNING job's state and allocation until the simulator confirms its current run has stopped; a failed stop releases nothing. Following valid completion under P3 or a confirmed stop, release exactly that run's allocation after matching both run ID and allocation generation against the node's current owner. A mismatch, duplicate or delayed notification must leave any newer allocation unchanged; handling the same terminal notification again must have no new effect. For example, Master's old notification must not release a node reassigned to a newer PhD run. Role permissions are defined in P1. [source]

P6.1 Release only a confirmed current allocation.

Adapter. Record confirmed stop/completion results before updating policy state.

P6.2 Require a bounded nonblank reason and retain it with the stop.

Reuses P1.3, P1.2.

Adapter. Original validation checks reason text; the helper records StopReason.

Partial; see Section 2.2.

P6.3 Keep RUNNING state and allocation until the local runner confirms stop.

Adapter. A failed stop returns ok=False; state and allocation remain unchanged.

P6.4 Match current run ID and allocation generation before release.

Reuses P3.5, P4.7.

Adapter. Original Store checks allocation tokens; the helper records the release.

P6.5 Ignore stale/duplicate completion and make terminal stop acknowledgement harmless.

Adapter. Duplicate terminal stops return ALREADY_TERMINAL without new fact edits.

P6.6 An old Master notification must not release a newer PhD allocation.

Adapter. See P6.4 and P6.5.

P6.7 Use the P1 owner/Professor stop permissions.

Reuses P1.3, P1.2.

Adapter. See P1.

2.2 Unsupported and conditional scope

Output and IFC (P2.1–P2.3). Original view/read_private filter outputs; QueueViewer/PrivateReadable do not. General message/context IFC is missing: T2 UNSAFE_PEER_NOTE still leaks. Only structured completion reports are guarded.

Trace and secrets (P2.6–P2.7). Evaluator-only trace access relies on deployment. Credential and secret-flow checks are not implemented.

Observation and commits (P3.4, P4.7). Store and EDB commit separately; synchronization failures stop the scope. Restart reconciliation and external drift detection are not implemented.

Scheduling scope (P5.1, P5.4). Aging constrains dispatch choices, not future progress. Waiting updates assume the pinned one-node-per-type backend; other schedulers need explicit change records.

Reason semantics (P6.2). Reasons are checked for nonblank text and length, not justification.

Trusted application boundary (All). Host role strings are not authentication. No real runner is integrated; the T4 direct Store bypass remains unguarded.

Source: pinned student repository.

3 Catch Me If You Scam

3.1 Policy walkthrough

Grey boxes quote original P1–P6. Pn.m are our decomposition; rules follow in source order.

P1

Original policy P1. User Confirmation. An impostor may impersonate a trusted person and attempt to transfer funds without proper authorization. Therefore, every transfer must have explicit user confirmation for the exact recipient and amount. A confirmation must be associated with the specific transfer request. Instructions contained in external messages, including SMS messages, must not be treated as user confirmation. [source]

P1.1 Treat impersonation as a threat, not a confirmation source.

Context only; see Section 3.2.

P1.2 Confirm this exact recipient and amount.

`TransferIntent(Intent, Recipient, Cents, Details)`: Required host input binding intent, recipient, cents and details.

`.key TransferIntent(Intent)`

`UserConfirmation(Intent, Recipient, Cents)`: Required authenticated host/UI input; absent from the original app.

`Purpose(Intent, Purpose)`: Host-bound transfer or verification intent.

`.key Purpose(Intent)`

`Consumed(Intent)`: Callback records successful consumption of an intent.

Adapted MCP tool-call. `execute_transfer(recipient, amount, transfer_request_id)`: Original transfer tool; the wrapper adds intent correlation metadata.

```
// Bind a real tool call to matching host intent and user confirmation; reject used intents.
ConfirmedRequest(Q, I) :- Request(Q, "payments", "execute_transfer"),
    Parameter(Q, "intent_id", I), Parameter(Q, "recipient", R), Parameter(Q, "amount_minor", C),
    TransferIntent(I, R, C, D), UserConfirmation(I, R, C), Purpose(I, "transfer"),
    not Consumed(I).
```

Partial; see Section 3.2.

P1.3 Bind confirmation to a specific immutable transfer intent.

Reuses P1.2.

Partial; see Section 3.2.

P1.4 Never turn message instructions into a user-confirmation fact.

Reuses P1.2.

Partial; see Section 3.2.

P2

Original policy P2. Additional Verification for Suspicious Recipients. A scammer often provides unfamiliar payment details and creates a sense of urgency to pressure the victim into transferring money. So, Transfers to unregistered recipients or recipients whose payment details originate from untrusted messages must not be executed without additional identity verification and a cooling-off period. A user's request to proceed does not, by itself, bypass these requirements. [source]

P2.1 Do not infer trust from familiar wording or urgency.

Context only; see Section 3.2.

P2.2 Require identity review and elapsed cooling for unknown or message-derived details.

`Payee(Id, Account)`: Reviewed .dl payees matching the mock bank.

```
// Reviewed configuration.
Payee("payee-1", "FAKE-ACCOUNT-0001").
```

`UntrustedDetails(Intent)`: The payment details may descend from an external message.

`IdentityVerified(Intent, Recipient, Cents)`: Required trusted identity-review result.

`CoolingUntil(Intent, Deadline)`: Host-supplied deadline; keyed by intent.

`.key CoolingUntil(Intent)`

`Clock(Now)`: Before hook records current server time in seconds.

`.key Clock()`

```
// Reviewed configuration.
Clock(0).

// A bank payee ID is registered.
Registered(R) :- Payee(R, Account).

// A bank-listed account number is also a registered recipient identifier.
Registered(R) :- Payee(Id, R).

// A transfer to an unregistered recipient requires additional review.
NeedsReview(I) :- TransferIntent(I, R, C, D), not Registered(R).

// External-message ancestry requires review even if the recipient is registered.
NeedsReview(I) :- UntrustedDetails(I).

// Ordinary registered details need no extra identity review, but still need confirmation.
ReviewSatisfied(I) :- TransferIntent(I, R, C, D), not NeedsReview(I).

// Reviewed exact details may proceed only after the trusted cooling deadline.
ReviewSatisfied(I) :- TransferIntent(I, R, C, D), NeedsReview(I), IdentityVerified(I, R, C),
    CoolingUntil(I, Until), Clock(Now), Until <= Now.
```

Partial; see Section 3.2.

P2.3 User confirmation cannot override review and cooling.

Reuses P3.3.

P3

Original policy P3. Daily Transfer Limit. A daily transfer limit can reduce potential financial losses if a user or an agent is deceived into making payments. The Payments Agent must not execute a transfer if the transfer amount, combined with the total amount transferred that day, exceeds the configured daily limit of $1,000. Transfers within the limit remain subject to all other applicable authorization requirements. [source]

P3.1 Cap possible daily loss.

Context only; see Section 3.2.

P3.2 Keep requested cents within the current remaining daily budget.

`DailyBudget(Remaining, ConfiguredLimit)`: Callback replaces remaining cents after a successful transfer.

`.key DailyBudget()`

```
// Reviewed configuration.
DailyBudget(90000, 100000).
```

`Balance(Cents)`: Callback replaces balance cents after a successful transfer.

`.key Balance()`

```
// Reviewed configuration.
Balance(500000).
```

```
// The EDB limit must be $1,000; successful effects maintain remaining integer cents.
WithinDailyLimit(Q) :- Request(Q, P, Event), Parameter(Q, "amount_minor", C),
    DailyBudget(Remaining, 100000), Balance(Available), C > 0, C <= Remaining, C <= Available.
```

Partial; see Section 3.2.

P3.3 A budget check alone never authorizes a transfer.

`KnownOrigin(Intent)`: At least one declared bank, user or external-message origin exists.

```
// Execute only the bound, confirmed, known-origin request after every independent check.
Allow(Q) :- ConfirmedRequest(Q, I), KnownOrigin(I), ReviewSatisfied(I), WithinDailyLimit(Q).
```

Reuses P1.2, P3.2.

P4

Original policy P4. Payment Information Provenance. Without tracking, information introduced by a scammer may be treated as real user-provided payment instructions. The system must track whether payment details originate from registered banking records, direct user input, or untrusted external messages. Rephrasing or repeating information from an untrusted message must not automatically change its recorded origin to trusted user input. [source]

P4.1 Do not let repeated instructions silently become trusted data.

Context only; see Section 3.2.

P4.2 Track bank, direct-user and external-message origins.

```
// A bank record is a recognized possible payment-detail origin.
KnownOrigin(I) :- TransferIntent(I, R, C, D), Provenance(D, "bank").
```

```
// Direct user input is recognized only when the trusted producer records that origin.
KnownOrigin(I) :- TransferIntent(I, R, C, D), Provenance(D, "user").
```

```
// External-message details are recognized as untrusted, not silently discarded.
KnownOrigin(I) :- TransferIntent(I, R, C, D), Provenance(D, "external_message").
```

Partial; see Section 3.2.

P4.3 A rewrite retains any declared external-message ancestor.

```
// Any declared external-message ancestor triggers review; adding user origin cannot erase it.
UntrustedDetails(I) :- TransferIntent(I, R, C, D), Provenance(D, "external_message").
```

Partial; see Section 3.2.

P5

Original policy P5. Separation of Agent Responsibilities. Separating responsibilities prevents an agent that processes untrusted messages from directly controlling financial transactions. Only the Payments Agent may invoke the transfer execution tool. The Assistant Agent may request payment verification but must not directly execute transfers or modify banking records. The verification should not result in a transfer execution. [source]

P5.1 Separate untrusted-message processing from payment execution.

Context only; see Section 3.2.

P5.2 Allow only the bound Payments principal to execute.

```
// A non-Payments principal is denied even if another rule accidentally permits the call.
Deny(Q, "payments_only") :- Request(Q, P, "execute_transfer"), P != "payments".
```

Existing MCP tool-call. `lookup_recipient(recipient_id)`: Look up a registered recipient.

```
// Only the bound Payments principal may use the original recipient lookup.
Allow(Q) :- Request(Q, "payments", "lookup_recipient"), Parameter(Q, "recipient_id", R).
```

Existing MCP tool-call. `check_daily_limit(amount)`: Check the remaining mock daily limit.

```
// Only Payments may inspect a budget check; the check result is derived in Datalog.
Allow(Q) :- Request(Q, "payments", "check_daily_limit"), Parameter(Q, "amount_minor", C).
```

Partial; see Section 3.2.

P5.3 Assistant verifies without a transfer or bank-write tool.

Reuses P5.2.

Partial; see Section 3.2.

P5.4 A verification-only request must not execute.

```
// A host-marked verification intent cannot be used to execute a transfer.
Deny(Q, "verification_only") :- Request(Q, P, "execute_transfer"), Parameter(Q, "intent_id", I),
    Purpose(I, "verification").
```

Partial; see Section 3.2.

P6

Original policy P6. Information Isolation Between Agents. Restricting information flow reduces unnecessary exposure of sensitive data and prevents malicious instructions embedded in external messages from crossing agent boundaries. The Assistant Agent must not access account balances or full bank account numbers directly. The Payments Agent must not receive the same message from the Assistant Agent, e.g., raw SMS content or unrelated conversation history. Communication between agents must be limited to the information necessary to verify and process the specific payment request. [source]

P6.1 Limit inter-agent disclosure to necessary payment information.

Context only; see Section 3.2.

P6.2 Keep balances and full accounts out of Assistant tools.

Existing MCP tool-call. `list_payees()`: Assistant lists payee names and masked accounts.

```
// The Assistant may request the public payee projection, never raw bank facts.
Allow(Q) :- Request(Q, "assistant", "list_payees").
```

Partial; see Section 3.2.

P6.3 Never forward raw SMS or unrelated history to Payments.

Partial; see Section 3.2.

P6.4 Send only structured fields required for this payment.

Partial; see Section 3.2.

3.2 Unsupported and conditional scope

Missing trusted producers (P1.2–P1.4, P2.2). Authenticated confirmation, identity-review and cooling producers are absent. Tests ingest synthetic host facts.

Missing lineage instrumentation (P4.2–P4.3). Tests supply Source/DependsOn; original message rewrites do not record data-version lineage.

Bank state and day boundary (P1.3, P3.2). Bank JSON and EDB commit separately. No restart reconciliation, drift detection or daily rollover. Policy uses integer cents; original bank arithmetic remains floating point.

Identity and separation (P5.1–P5.4). Principal checks require endpoint authentication. Free-text A2A verification is not bound to a transfer-safe scope. The original prompt rejects unregistered recipients outright.

Output and A2A scope (P6.2–P6.4). list_payees masks accounts; other outputs and raw request_payment_verification text lack a scoped structured bridge.

Motivating statements (P1.1, P2.1, P3.1, P4.1, P6.1). These sentences motivate policies; they do not define executable detectors.

Source: pinned student repository.